TDT4173 Modern Machine Learning in Practice

Group: [153] Italienerne
authors:
Riccardo Mazzoleni 152564
Simone Tolledi 151684
Samrath Singh Chhabra 152583

November 9, 2025

1 Introduction

We address the task of forecasting cumulative deliveries per `rm_id` from January 2nd to May 31st, 2025, using the provided datasets: `receivals.csv` (historical delivery records), `purchase_orders.csv` (planned future orders), and optional metadata (`materials.csv` and `transportation.csv`). The evaluation metric is the mean quantile loss at $\alpha = 0.2$, which penalizes overestimation four times more than underestimation. Consequently, the entire modeling design prioritizes conservative forecasts that avoid overpredicting future deliveries. For submission, we generate a cumulative trajectory for each `rm_id` and date, following the required competition format.

As an initial approach, we experimented with **LightGBM**, which is conceptually similar to XGBoost. While it handled large datasets efficiently and provided flexible tree-based predictions, it struggled to capture the long-term cumulative structure of the data and was sensitive to the extreme zero-inflation and heavy tails present in daily weights. After multiple trials, LightGBM did not offer substantial advantages over XGBoost, which proved slightly better in handling zero-heavy distributions and allowed for more robust quantile-based objectives with post-hoc calibration. We therefore shifted focus to evaluating two distinct modeling approaches in parallel.

The first of these was a standard **XGBoost** regressor trained jointly across all raw materials. This approach allowed for extensive feature engineering, including lagged and rolling signals, backlog summaries, and cross-material patterns, enabling the model to learn relationships across different `rm_ids`. However, being a tree-based model, it still faced challenges in explicitly capturing structured temporal components such as weekly or yearly seasonality, and it required careful postprocessing to enforce cumulative monotonicity.

In parallel, we explored a completely different approach with the **Prophet** model, trained separately for each *rm_id*. Prophet explicitly decomposes the signal into trend, seasonality, and holiday effects, making it naturally suited for modeling the structured temporal patterns of cumulative weights. As a result, Prophet became the primary model, particularly for the 30 most active materials based on delivery frequency and volume toward the end of 2024, while XGBoost served as a complementary, more generic alternative capable of leveraging cross-material information.

2 Data and Preprocessing

We start from the raw CSV files, parsing timestamps into dates and standardizing time zones when present. Receivals are aggregated into daily weights per `rm_id`, while purchase orders are parsed to extract planned arrival dates and quantities, aligned to daily buckets, and left-joined to the receivals table. Optional metadata, such as material class or transportation mode, are incorporated via `rm_id`.

Columns containing NaN values were removed, and identical timestamps per `rm_id` were merged by summing same-day receivals. A continuous daily index was constructed for each `rm_id`, explicitly filling missing days with zeros. The dataset was then split into pre-2025 history and the 2025 forecasting horizon, with the target defined as cumulative weight starting from January 1st, 2025.

To evaluate models while respecting temporal order and the cumulative nature of the target, we use an expanding-window protocol across multiple end dates from January to May 2025. Model selection is based on the average quantile loss at $\alpha = 0.2$ computed across all `rm_id`s and horizons. Hyperparameters are set to fixed values as detailed in the accompanying notebooks, and random seeds are fixed to guarantee full reproducibility.

3 Exploratory Data Analysis

We began the analysis by leveraging domain knowledge: receivals are mainly driven by logistics constraints and purchase schedules, while underestimation is generally safer for smelting operations. Activity levels are typically reduced on weekends and public holidays, and lead times vary depending on the transportation mode.

Figures 1 and 2 highlight the impact of calendar effects. The first covers two months of 2024 and shows that not only weekends but also national holidays such as Easter and May 1st produce a significant drop in deliveries. The second confirms that weekends throughout 2024 display no receivals at all. As discussed in Section 5.1, capturing these strong periodic and calendar-driven patterns proved considerably easier with the Prophet model than with XGBoost.

From a data-driven perspective, the distribution of daily weights is heavily zero-inflated and exhibits a heavy tail: most days record zero activity, interspersed with bursts of receivals. Receivals themselves are event-based arrivals, while purchase orders provide planned signals that may be delayed or only partially fulfilled, introducing additional variability.

Univariate exploration focused on daily weight per `rm_id`, cumulative curves, lengths of consecutive zero days, and monthly seasonality patterns. At a multivariate level, we examined correlations between receivals and order backlogs and compared patterns across material categories. We noticed that almost all `rm_id`s (except one) are linked to a single `product_id`, making it possible to directly associate the predicted product quantity with the corresponding material. We also observed that a few materials account for the majority of production, while many materials have few observations or remain largely inactive. This led to the decision to include only `rm_id`s that were active at least until October 2024 and had sufficiently rich histories for Prophet, which models each `rm_id` independently and cannot learn cross-material patterns. Fig.3 helped us to determine which `rm_id` to look for. For XGBoost, we instead attempted to let the model learn these cross-material relationships autonomously, helping it predict the many zero observations in the dataset.

The prevalence of zeros in daily weights motivated two different modeling strategies: one focusing on predicting daily weights with subsequent accumulation, and another modeling cumulative-from-start trajectories with monotonic enforcement. Data cleanup involved applying outlier clipping rules, handling missing purchase order dates, and normalizing across the calendar to ensure consistent temporal alignment. These steps provided a clear understanding of the underlying structure and variability, guiding subsequent modeling and feature engineering.

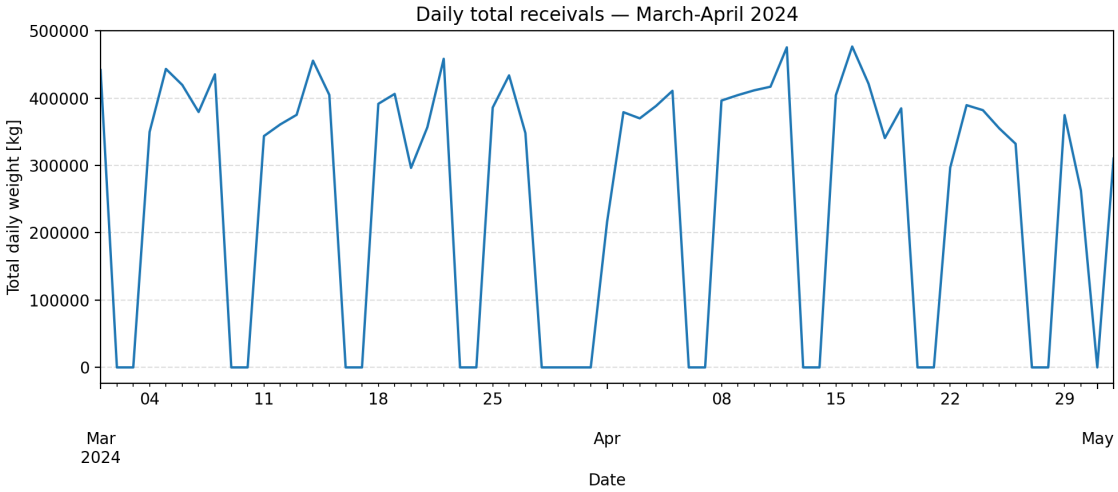

Figure 1: Daily total receivals (sum across all `rm_id`) from March 1st to May 2nd, 2024.

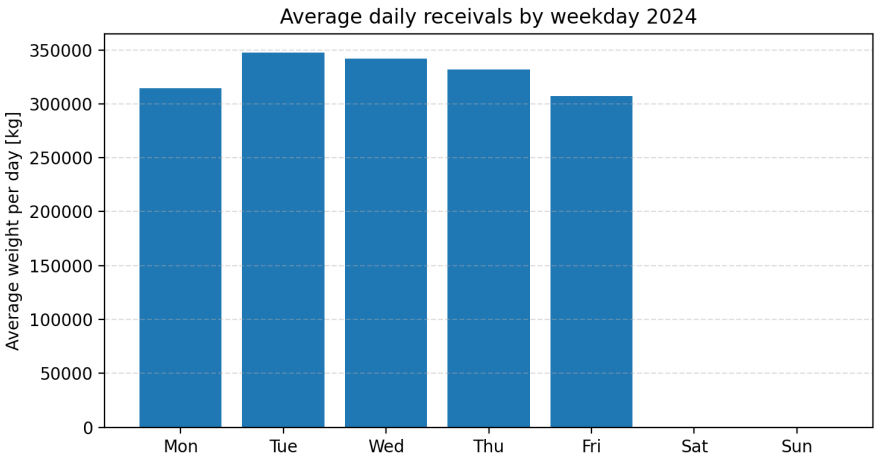

Figure 2: Average daily receivals by weekday in 2024. Similar values over working days while zero values over weekends which indicates no activity.

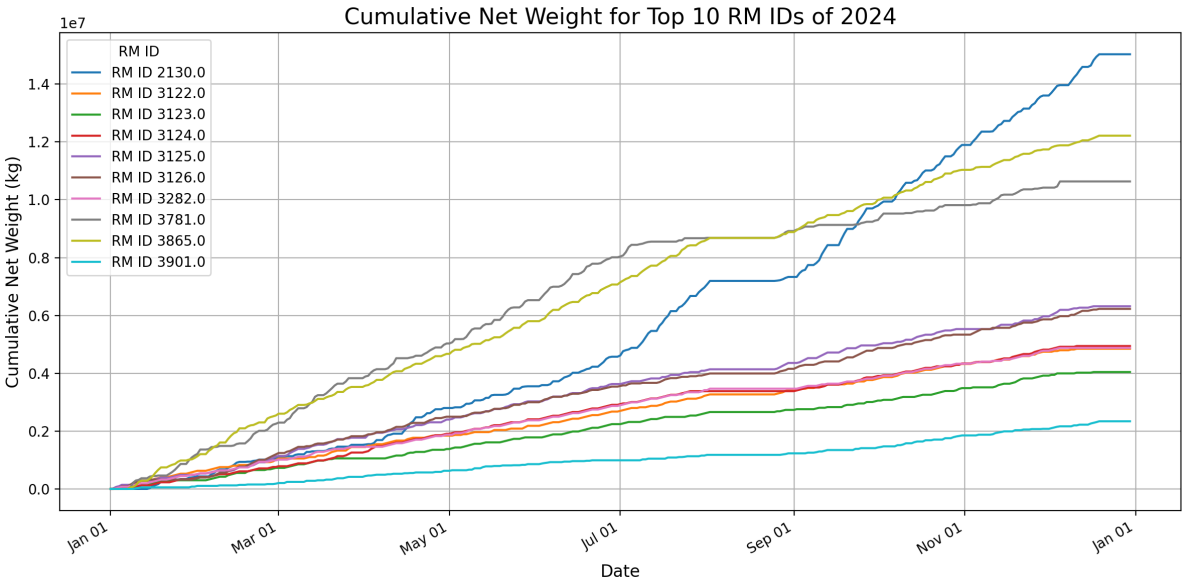

Figure 3: Cumulative raw materials in the first 5 months of 2024.

4 Feature Engineering

We performed extensive feature engineering to provide both models with informative inputs, though the requirements differ between Prophet and XGBoost. Prophet, being an additive time-series model, can handle temporal patterns more autonomously, especially seasonality and calendar effects, and therefore requires fewer engineered features. XGBoost, in contrast, benefits from a richer feature set to capture nonlinear interactions and short-term dynamics across `rm_id`s.

Calendar and cyclic time features were created for both models. We encoded cyclic patterns using sin/cos transformations for day-of-week and month, and included binary flags for weekends and public holidays. These features help align forecasts with recurring operational closures.

Lagged and rolling signals were computed per `rm_id`. These include lags of daily weight (e.g., 1, 7, 14, 28 days), rolling sums, means, and maxima over multiple windows (7, 14, 28 days), as well as the length of zero-streaks since the last non-zero observation. For purchase orders, we aggregated planned quantities over upcoming 7, 14, and 28-day windows, and computed overdue backlog. Overly aggressive lag stacks (more than 30-day lags) in XGBoost were found to induce overfitting and reduce generalization. These features provide XGBoost with information about recent momentum and short-term supply dynamics, which is less critical for Prophet due to its explicit modeling of seasonality and trend.

We also explored incorporating categorical identifiers such as `product_id` and `rm_id` directly as features in XGBoost to capture material-specific patterns. For Prophet, each `rm_id` was modeled separately, making such identifiers unnecessary.

Target construction differed slightly between models. For XGBoost, the model predicts daily weight, which is then accumulated to produce the required cumulative output. Alternatively, we explored directly modeling the cumulative-from-start trajectory, with monotonicity enforced in postprocessing (Section 7) to ensure realistic non-decreasing forecasts.

Overall, this systematic feature engineering ensured that XGBoost could leverage both short-term signals and material-specific information, while Prophet relied primarily on its internal trend, seasonal, and calendar decomposition, requiring fewer external features.

5 Models selection

5.1 Primary: Prophet

Each `rm_id` was modeled independently using Prophet, with the objective of forecasting its total cumulative weight over time. The model relied on additive components to capture the temporal structure and known calendar effects. Both yearly and weekly seasonalities were enabled, while changepoints were tuned to accommodate possible regime changes, such as changes in logistics or production schedules. Additional regressors included weekend and holiday indicators to reflect systematic slowdowns in activity. We also experimented with purchase_order–derived daily features, but these did not produce consistent improvements.

5.2 Alternative: XGBoost

Unlike Prophet's per-`rm_id` training, the XGBoost model was trained globally on the full feature matrix, where `rm_id` was included as a categorical feature alongside temporal and calendar variables. This allowed the model to share statistical strength across materials while capturing

cross-material patterns. The regressor was tuned over tree depth (6–8), learning rate (0.03–0.1), and number of trees (500–1500), with subsampling and column sampling for regularization. We tested both quantile objectives (with $\alpha = 0.2$) and a standard squared-error setup followed by post-hoc quantile calibration. Quantile objectives without post-hoc smoothing resulted in noisy tails, showing that some form of postprocessing is beneficial for stable forecasts.

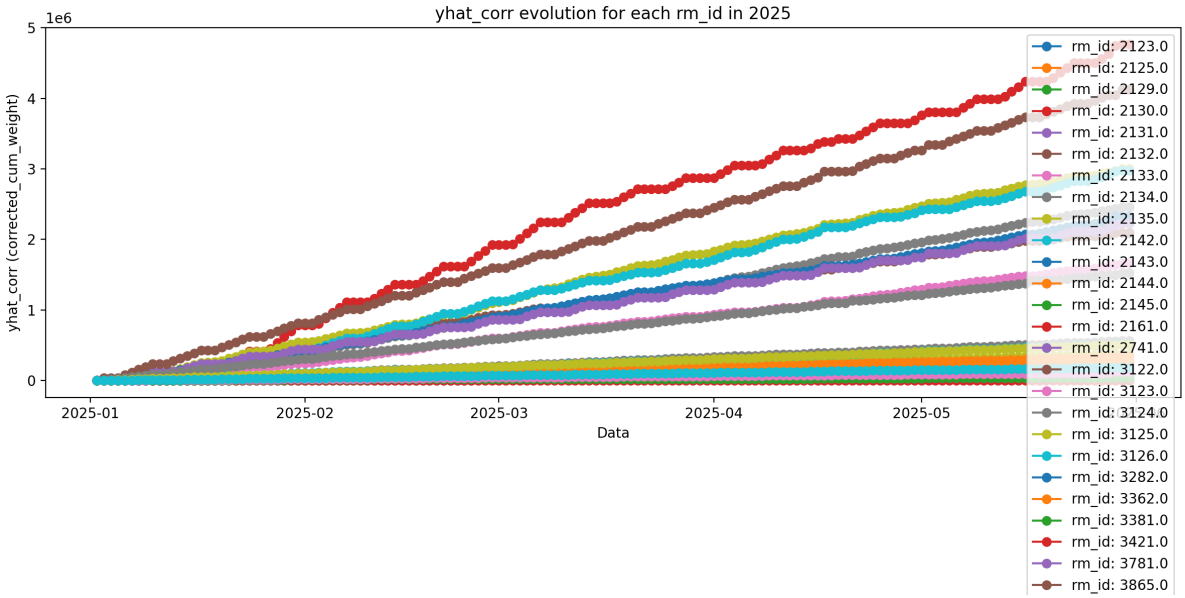

Figure 4: Cumulative daily weight predictions for the 2025 of the 30 raw materials trained with the Prophet model.

6 Model Interpretation

6.1 Prophet components

Prophet decomposes the cumulative weight time series into a smooth trend component, a set of seasonal components, and additive effects from external regressors. The trend captures long-term growth or decline patterns, while the seasonalities model recurring fluctuations within fixed periods.

In our case, the weekly seasonality and the working-days regressor play a central role: they clearly highlight the recurring production shutdowns during weekends and holidays, allowing the model to align forecasts with the actual operational calendar. These components dominate the short-term dynamics and substantially improve interpretability, making it clear when no activity is expected.

Changepoints further capture abrupt variations in the underlying trend, such as sudden logistics or scheduling shifts. On the other hand, including continuous regressors like the annual production quantity had limited impact: since Prophet combines effects additively, such variables mainly shift the overall level of the forecast rather than interacting meaningfully with seasonality or trend components. We also experimented with interaction-like terms, such as the product between annual quantity and day of year, but these did not yield noticeable improvements — further confirming that Prophet's additive framework is not well suited to capture multiplicative or time-varying intensity effects.

6.2 XGBoost interpretability

As an alternative to Prophet, XGBoost was applied as a global gradient-boosted trees model trained over the complete feature set, with `rm_id` encoded as a categorical feature. This approach captures nonlinear interactions and threshold effects that additive models like Prophet cannot, providing a more flexible and data-driven framework.

Feature importance metrics (gain and cover) reveal that backlog-related variables and recent rolling aggregates play the most influential role, effectively summarizing short-term production momentum.

Although XGBoost can model complex dependencies, it does not explicitly disentangle trend, seasonality, and calendar effects as Prophet does. The model also proved sensitive to the COVID-19 period, which had to be excluded from training to avoid downward-biased predictions. Moreover, hyperparameter optimization and validation are considerably slower than for Prophet, making rapid iteration more difficult. Consequently, Prophet remained our main model due to its transparency and efficiency, while XGBoost provided a valuable complementary benchmark for assessing nonlinear effects.

7 Postprocessing and Submission

After generating the raw forecasts, we applied a domain-informed postprocessing layer to ensure that the final predictions were realistic, consistent and aligned with operational constraints. Daily forecasts were first clipped to enforce non-negativity, preventing the model from producing implausible negative weights. The cumulative profiles were then reconstructed as non-decreasing series by computing the cumulative sum of these non-negative daily predictions. Direct cumulative regression without this monotonic enforcement had previously led to non-monotone curves, highlighting the importance of explicitly imposing a non-decreasing constraint for realistic cumulative forecasts..

Since Prophet is primarily optimized to minimize mean squared error, it does not inherently penalize asymmetric risks between overestimation and underestimation. To correct for this, we introduced a multiplicative shrinking factor to recalibrate the forecasts. This factor was designed to vary proportionally to the 2025 annual quantity, which we interpret as an indicator of reliability (larger annual quantities imply a higher likelihood that a given `rm_id` will actually be produced). We initially attempted to learn these shrinking factors by minimizing the pinball loss with $\alpha = 0.2$, aiming to penalize overestimation more strongly. However, when fitted on the training set, this approach produced overly optimistic correction factors (as expected, since the model was already tuned on that data), and splitting into separate training and test subsets led to unstable results due to frequent, abrupt changes in production trends.

Given these limitations, we adopted a more conservative strategy: choosing shrinking factors that deliberately lean toward underestimation. In our operational context, overstating future production is considerably more detrimental than providing cautious, slightly lower estimates. Finally, the recalibrated forecasts were formatted for submission, organized by `rm_id` and by date (from January 2nd to May 31st, 2025).

8 Conclusions

Prophet, complemented by conservative postprocessing and calendar regressors, delivers robust and interpretable cumulative forecasts under the asymmetric quantile loss. XGBoost serves as a strong alternative and a valuable stress test for feature design, confirming the relevance of

lag-based and backlog-related predictors.

The final pipeline is simple to reproduce, cautious against overestimation, and transparent through both component- and feature-level interpretations. Future extensions could involve combining the two approaches, for instance using Prophet-generated trend and seasonality components as inputs to XGBoost, or integrating external regressors such as weather or transport disruptions to further improve forecasting accuracy.